



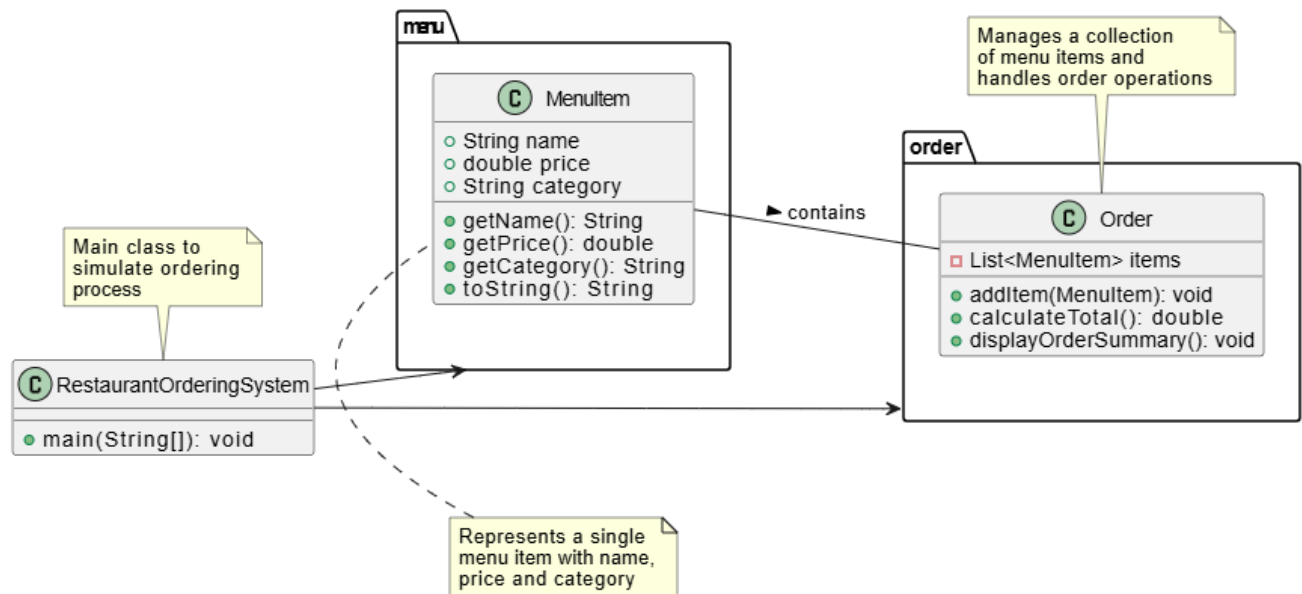
Open Ended Lab

Submitted By	Ijlal Ahmad
Submitted To	Dr. Jaweria Kanwal
Roll No	FL-21565
Date	09-05-2025
Subject	SCD

Scenario: Mini Restaurant Ordering System

✓ Task 1: Design

Package Diagram



✓ Task 2: Implementation

1.

```
1 package menu;
2
3 /*
4  LEGAL NOTICE:
5      * Author: YourName
6      * Course Project: Mini Restaurant Ordering System
7      * Version: 1.0
8      * Date: May 2025
9      * License: For educational use only. Do not distribute.
10 */
11
12
13 // Represents a menu item in the restaurant // INFORMATIVE
14 public class MenuItem {
15     private String name;
16     private double price;
17     private String category;
18
19     // Constructor to create a new menu item // INFORMATIVE
20     public MenuItem(String name, double price, String category) {
21         this.name=name;
22         this.price=price;
23         this.category=category;
24     }
25
26     // Getter methods to access private variables. // INFORMATIVE
27     public String getName() {
28         return name;
29     }
30
31     public double getPrice() {
32         return price;
33     }
34
35     public String getCategory() {
36         return category;
37     }
38
39     public String toString() {
40         return category+": "+name + "- $" + price;
41     }
42
43 }
44
```

2.

```
1 package order;
2
3 import menu.MenuItem;
4
5
6
7 //Author: Ijlal Ahmad | Handles customer orders in the system. //Legal Comment
8
9
10 public class Order {
11     private List<MenuItem> items;
12
13
14     public Order() {
15         items = new ArrayList<>();
16     }
17
18
19     public void addItem(MenuItem item) {
20         items.add(item);
21     }
22
23
24     public double calculateTotal() {
25         double total = 0;
26         for (MenuItem item : items) {
27             total += item.getPrice(); //shows summation logic // Amplification
28         }
29         return total;
30     }
31
32
33     public void displayOrderSummary() {
34
35         System.out.println("=== Order Summary ===");
36         for (MenuItem item : items) {
37             System.out.println(item);
38         }
39
40         System.out.println("=====");
41     }
42
43
44     public List<MenuItem> getItems() {
45         return items;
46     }
47 }
```

3.

```
1 package restaurant;
2 import menu.MenuItem;
3
4
5
6
7
8 public class RestaurantOrderingSystem {
9     public static void main(String[] args) {
10
11         // Create menu items // Informative
12         MenuItem item1 = new MenuItem("Spring Rolls", 5.99, "Appetizer");
13         MenuItem item2 = new MenuItem("Chicken Curry", 12.49, "Main Course");
14         MenuItem item3 = new MenuItem("Lemonade", 2.50, "Drink");
15
16         // Create a new order //Informative
17         Order order = new Order();
18         order.addItem(item1);
19         order.addItem(item2);
20         order.addItem(item3);
21
22         // Display the order summary //Informative
23         order.displayOrderSummary();
24     }
25 }
26
```

Output

```
==== Order Summary ====
Appetizer: Spring Rolls- $5.99
Main Course: Chicken Curry- $12.49
Drink: Lemonade- $2.5
=====
```

✓ Task 3: Code Quality and Documentation

- 5 clean coding practices

1. camelCasing:

I use camelCasing in my code in which the first letter of first word is small and the first letter of rest of the words are capital.

For Example:

```
getName
addItem
displayOrderSummary
```

2. Intention revealing names:

I used the names for constructors, functions etc which clearly reveals its function that what it's for.

For example:

I used MenuItem (which clearly represent a food item)

calculateTotal() (exactly describe that it calculate the total price of items)

3. Single Responsibility Principle (SRP):

have only one responsibility.

Practice: Each class/method should

For Examples:

MenuItem class: Only stores item details (name/price/category)

Order class: Manages order operations (add items/calculate totals)

calculateTotal() only sums prices, doesn't handle display logic

4. Mental mapping:

Instead of creating abbreviation for names, I used the full name and I avoid the mental mapping so if someone read my code, he/she may not get confused.

5. Encapsulation:

Restrict direct access to class fields.

For Example:

```
Private String name;  
Private double price;  
Private String category;  
private List<MenuItem> items;
```

This prevents external code from modifying price or items directly, which could corrupt data.

• Justifying comments

LEGAL Comments:

These comments is useful for identifying who created the file, what version it is, and any licensing or usage restrictions.

INFORMATIVE Comments:

Informative comments help others (or your future self) quickly understand the purpose of a line or block of code without needing to reverse-engineer the logic.

AMPLIFICATION Comments:

An amplification comment explains why something is done in the code, not just *what* it does.

TODO comments:

TODO comments act like sticky notes in your code. They remind developers what needs to be implemented or fixed later